

Phase 4 Structured Notes: Why Red-Black Trees and 2-3-4 Trees Are the Same Idea

0. Big Idea of Phase 4

Phase 4 teaches the connection between **2-3-4 Trees** and **Red-Black Trees**.

The main idea is:

A Red-Black Tree is a 2-3-4 Tree drawn as a binary tree.

A **2-3-4 Tree** can store multiple keys inside one node.

A **Red-Black Tree** stores only one key in each physical node, but it uses **red nodes** to show which keys should be grouped together.

So the core memory rule is:

Black node = starts a real 2-3-4 node

Red node = extra key inside the same 2-3-4 node as its black parent

This phase helps you understand why Red-Black Tree insertion uses recoloring and rotations. They are not random rules. They are the binary-tree version of 2-3-4 Tree splitting and rearranging.

1. Current Progress Before Phase 4

Before Phase 4, you have already learned three major ideas.

1.1 Phase 1: 2-3 Trees

A **2-3 Tree** is a balanced multi-way search tree.

A 2-3 Tree node can store:

- 1 key, or
- 2 keys

A 2-3 Tree stays balanced because:

All leaves stay at the same level.

You also learned:

- insertion happens at a leaf
- if a node becomes too full, it splits
- if deletion creates an empty node, borrow first
- if borrowing is impossible, merge

1.2 Phase 2: 2-3-4 Trees

A **2-3-4 Tree** is a wider version of a 2-3 Tree.

A 2-3-4 Tree node can store:

- 1 key
- 2 keys
- 3 keys

A node with 3 keys is called a **4-node** because it can have 4 child ranges.

The main Phase 2 rule was:

A 2-3-4 Tree is balanced because all leaves stay at the same level.

You also learned that when a 2-3-4 node temporarily gets 4 keys, it must split.

1.3 Phase 3: Red-Black Trees

A **Red-Black Tree** is a Binary Search Tree with color rules.

Each node stores:

- data
- color: red or black
- left child pointer
- right child pointer
- parent pointer

The key Phase 3 ideas were:

- new nodes start red
- root must be black
- null children count as black
- every root-to-null path must have the same black count
- red nodes cannot have red parents or red children
- red-red conflicts are fixed by recoloring or rotations

The main Red-Black insertion rule was:

Red uncle means recolor.
Black or null uncle means rotate.

1.4 Why Phase 4 matters

Red-Black Trees can feel confusing if you only memorize their rules.

Phase 4 explains the reason behind those rules.

The key idea is:

Red-Black Trees are just 2-3-4 Trees written in binary-tree form.

So after Phase 4, recoloring and rotations should feel more logical.

2. The Core Mapping Rule

The most important Phase 4 rule is this:

A black node represents a real 2-3-4 node.
A red node represents an extra key inside its black parent's 2-3-4 node.

In simple words:

- Black nodes are the main nodes.
- Red nodes are attached keys.
- A red child is mentally grouped with its black parent.

2.1 Black node meaning

A black node means:

This key starts a separate 2-3-4 node.

Example:

20B

This represents the 2-3-4 node:

[20]

2.2 Red node meaning

A red node means:

This key belongs inside the same 2-3-4 node as its black parent.

Example:

20B

40R

This represents:

[20, 40]

The red node 40R is not treated as a separate 2-3-4 node. It is grouped with 20B.

2.3 One-sentence version

Black means separate. Red means grouped with parent.

3. Mapping 2-3-4 Nodes into Red-Black Form

A 2-3-4 Tree has three possible node types:

2-3-4 node type	Number of keys	Red-Black form
2-node	1 key	One black node
3-node	2 keys	One black node with one red child
4-node	3 keys	One black middle key with two red children

Let us study each one carefully.

4. Case 1: A 2-Node

4.1 What is a 2-node?

A **2-node** has 1 key.

Example in 2-3-4 form:

```
[20]
```

This node creates two child ranges:

```
values < 20  
values > 20
```

4.2 Red-Black form of a 2-node

The Red-Black form is:

```
20B
```

`20B` means:

- key = 20
- color = black

4.3 Meaning

A single black node represents one normal 2-3-4 node with one key.

So:

```
2-3-4:    [20]  
Red-Black: 20B
```

4.4 Beginner memory

One key in a 2-3-4 node becomes one black Red-Black node.

5. Case 2: A 3-Node

5.1 What is a 3-node?

A **3-node** has 2 keys.

Example in 2-3-4 form:

[20, 40]

This node creates three child ranges:

```
values < 20  
20 < values < 40  
values > 40
```

5.2 Red-Black form of a 3-node

A 3-node becomes one black node with one red child.

One possible drawing is:

20B

40R

This means:

$20B + 40R = [20, 40]$

The red node **40R** is grouped with the black parent **20B**.

5.3 Another possible drawing

A 3-node can also be drawn with the red child on the left:

```
  40B
 /
20R
```

This also represents:

```
[20, 40]
```

Both drawings represent a 2-key 2-3-4 node.

5.4 Why both drawings can work

Both drawings keep the BST order correct:

```
20 < 40
```

The important rule is not whether the red child is left or right.

The important rule is:

The red child belongs inside the same 2-3-4 node as the black parent.

5.5 Beginner memory

A 3-node becomes a black node with one red child.

6. Case 3: A 4-Node

6.1 What is a 4-node?

A **4-node** has 3 keys.

Example in 2-3-4 form:

```
[10, 20, 30]
```

This node creates four child ranges:

```
values < 10
10 < values < 20
20 < values < 30
values > 30
```

6.2 Red-Black form of a 4-node

A 4-node is represented as a black middle key with two red children.

```
  20B
 /
10R  30R
```

This means:

```
[10, 20, 30]
```

6.3 Why the middle key is black

The middle key is the best root of the small binary shape because it keeps the search order balanced:

```
10 < 20 < 30
```

So:

- 20 becomes the black parent
- 10 becomes the red left child
- 30 becomes the red right child

6.4 What the red children mean

The red children do not represent separate 2-3-4 nodes.

They are extra keys inside the same 2-3-4 node.

So this Red-Black shape:


```
    20B
  /
10R  30R
```

should be mentally compressed into:

```
[10, 20, 30]
```

6.5 Beginner memory

A 4-node becomes a black middle key with two red children.

7. Quick Mapping Summary

7.1 2-node mapping

```
2-3-4 form:
[20]
```

```
Red-Black form:
20B
```

7.2 3-node mapping

```
2-3-4 form:
[20, 40]
```

```
Red-Black form:
20B
```

```
    40R
```

7.3 4-node mapping

```
2-3-4 form:
[10, 20, 30]
```

```
Red-Black form:
```

```
    20B
   /
 10R  30R
```

7.4 Main translation rule

When reading a Red-Black Tree:

1. Find a black node.
2. Attach its red children to it mentally.
3. Compress that group into one 2-3-4 node.
4. Treat black children as separate 2-3-4 nodes.

8. Why New Red-Black Nodes Start Red

In Phase 3, you learned:

New Red-Black nodes start red.

Phase 4 explains why.

8.1 The 2-3-4 Tree reason

In a 2-3-4 Tree, when we insert a new key, we first try to add it into an existing node.

Example:

```
[10]
```

Insert **20**:

```
[10, 20]
```

The tree did not grow taller.

We simply added another key into the same node.

8.2 Red-Black version

The Red-Black version is:

10B

20R

The red node 20R means:

20 is grouped with 10.

It is like saying:

$10B + 20R = [10, 20]$

8.3 Why not insert new nodes as black?

If a new node were inserted as black, it would create a new separate 2-3-4 node immediately.

That could increase black height on one path and break the Red-Black rules.

Red is safer because red does not increase black depth.

8.4 Simple explanation

A new red node means:

Try to fit this new key into an existing 2-3-4 node first.

Only if that causes a problem do we repair the tree.

8.5 Beginner memory

New nodes start red because red means “extra key inside an existing 2-3-4 node.”

9. Recoloring Means Splitting

One of the most important Phase 4 ideas is:

Red-Black recoloring is the binary version of 2-3-4 splitting.

9.1 Splitting in a 2-3-4 Tree

A 2-3-4 node can hold at most 3 keys.

Example:

[10, 20, 30]

Now insert 50.

Temporary overflow:

[10, 20, 30, 50]

This is not allowed permanently.

So the node splits.

One way to understand the split:

[10, 20, 30, 50]

splits into:

 [20]
 /
[10] [30, 50]

The key 20 moves upward.

9.2 Same situation in Red-Black form

Before the problem is fixed, the Red-Black shape may look like this:

 20B
 /
10R 30R
 50R

There is a red-red conflict:

30R -> 50R

The parent is red.

The uncle is also red:

10R

So the Red-Black insertion rule says:

Parent red and uncle red means recolor.

9.3 Recoloring steps

Before:

```
    20B
   /
 10R  30R
      50R
```

Recolor:

10R becomes black
30R becomes black
20B becomes red, unless 20 is the root

If 20 is the root, it must stay black.

Result idea:

```
    20B
   /
 10B  30B
      50R
```

9.4 Why this matches splitting

In the 2-3-4 Tree, a full node split and pushed a key upward.

In the Red-Black Tree, recoloring also pushes the problem upward.

So:

2-3-4 Tree: full node splits
Red-Black Tree: parent and uncle recolor

9.5 Beginner memory

Recoloring means the 2-3-4 node was too full and had to split.

10. Rotation Means Rearranging the Binary Drawing

The second important Phase 4 idea is:

Red-Black rotation rearranges the binary drawing so it correctly represents the 2-3-4 node.

10.1 The 2-3-4 Tree view

Insert:

10, 20, 30

In a 2-3-4 Tree, these can fit into one node:

[10, 20, 30]

This is valid because a 2-3-4 node can hold 3 keys.

10.2 The first Red-Black drawing

If we insert these keys using normal BST logic first, we get:

10B

20R

30R

This has a red-red conflict:

20R -> 30R

The uncle is black or null.

So the Red-Black rule says:

Parent red and uncle black/null means rotate.

10.3 After rotation

After rotation, we get:

20B
/
10R 30R

Now this Red-Black shape correctly represents:

[10, 20, 30]

10.4 What rotation really does

Rotation does not change the sorted order.

Before rotation, the sorted order was:

10 < 20 < 30

After rotation, the sorted order is still:

```
10 < 20 < 30
```

Rotation only changes the shape.

10.5 Simple explanation

A rotation says:

The keys are correct, but the binary drawing is awkward. Rearrange it so the grouped 2-3-4 node is represented cleanly.

10.6 Beginner memory

Rotation changes shape, not order.

11. Recoloring vs Rotation

Red-Black insertion uses two main repair tools:

- recoloring
- rotation

Phase 4 explains what each tool means in 2-3-4 language.

Red-Black action	2-3-4 meaning	Simple meaning
Recoloring	Splitting	A 2-3-4 node became too full
Rotation	Rearranging grouped keys	The binary drawing needs a better shape

11.1 When recoloring happens

Recoloring happens when:

```
parent is red  
uncle is red
```

This usually means the current 2-3-4 node was full and must split.

11.2 When rotation happens

Rotation happens when:


```
parent is red
uncle is black or null
```

This usually means the tree needs to rearrange the binary shape.

11.3 Memory rule

```
Red uncle -> recolor -> split
Black/null uncle -> rotate -> rearrange
```

12. General Formula from Phase 4

This is a useful way to connect both trees.

Concept	2-3-4 Tree	Red-Black Tree
New separate branch	Move to a separate child node	Child is black
Extra key inside current node	Add key to current 2-3-4 node	Child is red
Node overflow	Split node and push key up	Recolor and maybe rotate
Key order correction	Rearrange keys inside multi-key node	Rotate binary nodes
Cascading split	Split may move upward	Recolor/rotation may move upward

12.1 New branch means black

If a key belongs in a separate 2-3-4 child node, the Red-Black version uses a black child.

12.2 Internal key means red

If a key belongs inside the same 2-3-4 node, the Red-Black version uses a red child.

12.3 Split means recolor or rotation

If a 2-3-4 node overflows, the Red-Black version fixes it using recoloring and possibly rotation.

13. Side-by-Side Insertion Dry Run

Now let us insert these keys:

10, 20, 30, 50, 40, 60, 70, 80

This dry run shows the same tree being built in two different languages:

- 2-3-4 Tree language
 - Red-Black Tree language
-

13.1 Insert 10

2-3-4 Tree

The tree is empty.

Insert 10:

[10]

Red-Black Tree

Insert 10 as red first, but the root must be black.

10B

Meaning

Both trees have one key.

[10] = 10B

13.2 Insert 20

2-3-4 Tree

The root has space.

[10, 20]

Red-Black Tree

Insert 20 as a red child:

10B

20R

Meaning

The red node is grouped with the black parent.

$10B + 20R = [10, 20]$

Rule shown

A red child under a black parent is allowed.

13.3 Insert 30

2-3-4 Tree

The node still has space for one more key.

[10, 20, 30]

This is a valid 4-node.

Red-Black Tree before repair

BST insertion first gives:

10B

20R

30R

There is a red-red conflict:

20R -> 30R

The uncle is null, so it counts as black.

Red-Black repair

This is an RR case, so we rotate left around 10.

After repair:

```
  20B
 /
10R  30R
```

Meaning

The Red-Black Tree now represents:

[10, 20, 30]

Rule shown

Rotation makes the binary drawing match the 2-3-4 node.

13.4 Insert 50

2-3-4 Tree before split

Current node:

[10, 20, 30]

Insert 50 temporarily:

[10, 20, 30, 50]

This is too many keys.

2-3-4 Tree split

The node splits:

```
    [20]
   /
[10] [30, 50]
```

Red-Black Tree before repair

Before repair, the shape has parent and uncle red:

```
    20B
   /
10R  30R
     /
    50R
```

The problem is:

30R -> 50R

The uncle is:

10R

The uncle is red.

Red-Black repair

Red uncle means recolor.

10R becomes 10B
30R becomes 30B
20B may become red, but root must stay black

Result idea:

```
    20B
   /
```

10B 30B
50R

Meaning

The 2-3-4 split and the Red-Black recoloring are the same idea.

Rule shown

Recoloring = splitting

13.5 Insert 40

2-3-4 Tree

40 belongs with the node:

[30, 50]

Insert 40:

[30, 40, 50]

This is valid because a 2-3-4 node can hold 3 keys.

Red-Black Tree

In Red-Black form, inserting 40 creates a zig-zag shape around 30, 50, and 40.

The repair uses rotation.

After rotation, the grouped node becomes:

40B
/
30R 50R

Meaning

This represents:

```
[30, 40, 50]
```

Rule shown

Rotation rearranges the Red-Black drawing so the 2-3-4 group is clear.

13.6 Insert 60

2-3-4 Tree

Current grouped node:

```
[30, 40, 50]
```

Insert 60 temporarily:

```
[30, 40, 50, 60]
```

This overflows.

2-3-4 Tree split

The node splits.

The key 40 moves upward.

The root becomes:

```
[20, 40]
```

The leaves include:

```
[10]  
[30]  
[50, 60]
```

Red-Black Tree

The Red-Black Tree sees parent and uncle red, so it recolors.

Meaning

Again:

Red-Black recolor = 2-3-4 split

Rule shown

Recoloring can push the balancing issue upward.

13.7 Insert 70

2-3-4 Tree

70 belongs with:

[50, 60]

Insert 70:

[50, 60, 70]

This is a valid 4-node.

Red-Black Tree

Red-Black uses rotation to represent the 4-node as:

60B
/
50R 70R

Meaning

This represents:

[50, 60, 70]

Rule shown

Rotation helps create the correct Red-Black shape for a 4-node.

13.8 Insert 80

2-3-4 Tree before split

Current grouped node:

[50, 60, 70]

Insert 80 temporarily:

[50, 60, 70, 80]

This overflows.

First 2-3-4 split

The lower node splits.

60 moves upward.

Cascading split

The parent/root receives too much.

So the root also splits.

Final 2-3-4 idea:

```
      [40]
     /
  [20]  [60]
     /
   [50]  [70, 80]
```

Red-Black Tree

The Red-Black Tree handles the same process using:

1. recoloring at the lower level
2. then another repair higher up
3. rotation/recoloring to keep the root correct

Meaning

This is the Red-Black version of a cascading 2-3-4 split.

Rule shown

2-3-4 cascading split = Red-Black repair moving upward

14. Full Side-by-Side Dry Run Table

Inserted key	2-3-4 Tree action	Red-Black Tree action	Main lesson
10	Create root [10]	Insert root and make it black	Root is black
20	Add to root: [10, 20]	Insert red child 20R	Red means grouped key
30	Root becomes [10, 20, 30]	Rotate to make 20B with red children	Rotation creates clean 4-node shape
50	[10, 20, 30, 50] overflows and splits	Parent and uncle red, so recolor	Recoloring equals splitting
40	Add to [30, 50] to make [30, 40, 50]	Rotate zig-zag shape	Rotation rearranges grouped keys
60	[30, 40, 50, 60] overflows and splits	Recolor	Recoloring pushes issue upward
70	Add to [50, 60] to make [50, 60, 70]	Rotate to make 60B with red children	4-node represented by black middle key
80	Lower split, then root split	Recolor, then higher repair	Cascading split equals upward Red-Black repair

15. How to Convert a Red-Black Tree into a 2-3-4 Tree

Use this method:

Step 1: Find black nodes

Every black node starts a 2-3-4 node.

Step 2: Attach red children

If a black node has red children, those red children are extra keys inside the same 2-3-4 node.

Step 3: Sort the grouped keys

Inside a 2-3-4 node, keys must be sorted.

Step 4: Black children become separate 2-3-4 child nodes

If a child is black, it starts a new 2-3-4 node.

Example

Red-Black form:

```
  40B
 /
20R  60R
```

Group red children with the black parent:

```
20R + 40B + 60R
```

Sorted:

```
[20, 40, 60]
```

So the 2-3-4 node is:

[20, 40, 60]

16. How to Convert a 2-3-4 Tree into a Red-Black Tree

Use this method:

16.1 For a 2-node

2-3-4 form:

[A]

Red-Black form:

AB

Meaning: A is black.

16.2 For a 3-node

2-3-4 form:

[A, B]

One possible Red-Black form:

AB

BR

Another possible form:

```
BB
/
AR
```

Both show one black key and one red grouped key.

16.3 For a 4-node

2-3-4 form:

```
[A, B, C]
```

Red-Black form:

```
BB
/
AR  CR
```

The middle key becomes black.

The smaller and larger keys become red children.

17. Relationship Between Red-Black Rules and 2-3-4 Trees

Red-Black rules make more sense when you see the 2-3-4 meaning.

17.1 Why no red-red conflict?

A red node means:

I am an extra key inside my black parent's 2-3-4 node.

If a red node has a red child, then an extra key has another extra key below it.

That would mean the 2-3-4 grouping is broken or too deep.

So Red-Black Trees do not allow red-red parent-child pairs.

17.2 Why same black depth?

Black nodes represent real 2-3-4 nodes.

In a 2-3-4 Tree, all leaves are at the same level.

In a Red-Black Tree, same black depth means:

Every path passes through the same number of real 2-3-4 nodes.

So the Red-Black black-depth rule is the binary version of the 2-3-4 leaf-level rule.

17.3 Why root is black?

The root must be a real 2-3-4 node.

Since black nodes represent real 2-3-4 nodes, the Red-Black root must be black.

17.4 Why nulls are black?

Null children mark the ends of paths.

Counting nulls as black helps measure black depth consistently.

17.5 Why new nodes are red?

New nodes start red because insertion first tries to add a key into an existing 2-3-4 node.

18. Red-Black Insertion in 2-3-4 Language

Red-Black insertion can be translated into 2-3-4 language.

18.1 Normal Red-Black insertion rule

1. Insert like a normal BST.
2. Color the new node red.
3. If parent is black, stop.
4. If parent is red, check uncle.
5. Red uncle -> recolor.
6. Black/null uncle -> rotate.
7. Force root black.

18.2 2-3-4 meaning

1. Try to add the key into an existing 2-3-4 node.
2. If the node has room, done.
3. If the node overflows, split it.
4. If the binary drawing is awkward, rotate to reshape it.
5. If the split moves upward, continue repairing upward.

18.3 Combined memory

Insert red = add key to current 2-3-4 node
Red uncle = split/recolor
Black uncle = rotate/rearrange

19. Conceptual Parallel Code

The idea can be written like this:

```
void insertCorresponding(int key) {  
    // 2-3-4 Tree idea  
    if (nodeIsFull()) {  
        splitNode();           // Push a key upward  
    } else {  
        addKeyToNode(key);     // Add inside the current node  
    }  
  
    // Red-Black Tree idea  
    insertAsRedNode(key);      // Try to group with parent  
  
    if (parentIsRed() && uncleIsRed()) {  
        recolor();             // Same idea as splitting  
    } else if (parentIsRed() && uncleIsBlackOrNull()) {  
        rotate();              // Rearrange binary shape  
    }  
}
```

19.1 What this code means

This code is not meant to be a full working tree library.

It is a teaching version.

It shows that the same insertion is happening in two different forms.

19.2 Simple explanation

In a 2-3-4 Tree:

- add key if there is space
- split if full

In a Red-Black Tree:

- insert red if possible
- recolor if the equivalent 2-3-4 node is full
- rotate if the binary shape needs fixing

20. Complete Teaching Program for Phase 4

This program prints a side-by-side trace.

It does not implement a full Red-Black Tree or 2-3-4 Tree.

Instead, it helps you see the matching actions.

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

struct Step {
    int key;
    string rbAction;
    string tree234Action;
};

int main() {
    vector<Step> steps = {
        {10,
         "Insert red; root becomes black.",
         "Create root [10]."},
        {20,
         "Insert red to the right of black 10; no red-red conflict."},
    };
}
```



```

        "Add to root -> [10, 20]."},

    {30,
        "Red-red conflict with parent 20; uncle is black/null; rotate. 20
becomes black with red 10 and 30.",
        "Add to root -> [10, 20, 30]."},

    {50,
        "Parent 30 and new 50 are red; uncle 10 is red; recolor.",
        "Overflow; split [10, 20, 30, 50] conceptually by pushing 20 up."},

    {40,
        "Red-red with parent 50; uncle black/null; rotate so 40 groups with 30
and 50.",
        "Add 40 to [30, 50] -> [30, 40, 50]."},

    {60,
        "Uncle 30 is red; recolor 30 and 50 black; 40 becomes red.",
        "Split [30, 40, 50, 60]; 40 moves up to root."},

    {70,
        "Uncle black/null; rotate so 60 is black with red 50 and 70.",
        "Add 70 to [50, 60] -> [50, 60, 70]."},

    {80,
        "First recolor at 70/80; then cascading red-red at 60/40; rotate so 40
is root.",
        "Split lower node, then split root; 40 becomes root."}
    };

cout << "Phase 4: Red-Black Tree and 2-3-4 Tree Parallel Creation\n\n";

for (const Step& s : steps) {
    cout << "Insert " << s.key << "\n";
    cout << "  Red-Black: " << s.rbAction << "\n";
    cout << "  2-3-4:      " << s.tree234Action << "\n\n";
}

cout << "Rule summary:\n";
cout << "  Red node = key inside the same 2-3-4 node as its black parent.
\n";
cout << "  Black node = separate 2-3-4 node.\n";
cout << "  Recolor = split movement.\n";
cout << "  Rotate = binary shape rearrangement.\n";

return 0;
}

```

20.1 What the program teaches

The program stores each insertion step in a `Step` object.

Each step has:

- the key being inserted
- the Red-Black action
- the matching 2-3-4 action

20.2 Why this program is useful

It helps you see that Red-Black Tree insertion and 2-3-4 Tree insertion are not separate ideas.

They are the same balancing process shown in two different ways.

20.3 What you should notice

When the 2-3-4 Tree adds a key to a node, the Red-Black Tree often inserts a red node.

When the 2-3-4 Tree splits, the Red-Black Tree often recolors.

When the Red-Black Tree rotates, it is reshaping the binary drawing of the same grouped keys.

21. Common Beginner Mistakes

Mistake 1: Thinking red means wrong

Wrong idea:

Every red node is bad.

Correct idea:

Red is allowed. Red-red is the problem.

A red node simply means the key is grouped with its black parent.

Mistake 2: Thinking a red child is a separate 2-3-4 node

Wrong idea:

Red child means a separate child node in the 2-3-4 Tree.

Correct idea:

A red child belongs inside the same 2-3-4 node as its black parent.

Mistake 3: Forgetting that black nodes start 2-3-4 nodes

A black node is not grouped into its parent in the same way.

A black node usually starts a separate 2-3-4 node.

Mistake 4: Mapping the wrong key as the black parent in a 4-node

For this 2-3-4 node:

```
[10, 20, 30]
```

The best Red-Black representation is:

```
  20B
 /
10R  30R
```

The middle key should be black.

Mistake 5: Treating recoloring and rotation as random rules

Recoloring and rotation are not random.

They have 2-3-4 meanings:

```
Recoloring = splitting
Rotation = rearranging binary shape
```

Mistake 6: Forgetting that rotations keep sorted order

Rotation changes the shape, but not the sorted order.

Example:

```
10 < 20 < 30
```

This order stays true before and after rotation.

Mistake 7: Forgetting that Red-Black Trees are still BSTs

Even with colors, a Red-Black Tree must still follow BST order:

```
left subtree < node < right subtree
```

Mistake 8: Thinking a 4-node has 4 keys

A 4-node has 3 keys and 4 child ranges.

Example:

```
[10, 20, 30]
```

This is a 4-node.

It does not have 4 keys.

Mistake 9: Forgetting duplicate keys are not allowed

Both 2-3-4 Trees and Red-Black Trees in this curriculum assume no duplicate keys.

Mistake 10: Thinking Phase 4 is a new algorithm

Phase 4 is mostly a mental bridge.

It explains why the algorithms from earlier phases make sense.

22. Key Vocabulary

2-3-4 Tree

A balanced multi-way search tree where one node can store 1, 2, or 3 keys.

Red-Black Tree

A balanced Binary Search Tree where each node has a color: red or black.

2-node

A 2-3-4 node with 1 key.

Example:

[20]

3-node

A 2-3-4 node with 2 keys.

Example:

[20, 40]

4-node

A 2-3-4 node with 3 keys.

Example:

[10, 20, 30]

Black node

In the Phase 4 mapping, a black node represents a separate 2-3-4 node.

Red node

In the Phase 4 mapping, a red node represents an extra key grouped with its black parent.

Recoloring

Changing node colors to fix a Red-Black Tree conflict.

In 2-3-4 language, recoloring usually represents splitting.

Rotation

A local rearrangement of a binary tree.

Rotation preserves BST order.

In 2-3-4 language, rotation helps rearrange grouped keys.

Red-red conflict

A Red-Black Tree violation where a red node has a red parent.

Uncle

The sibling of a node's parent.

The uncle's color decides whether insertion repair uses recoloring or rotation.

Split

A 2-3-4 Tree operation where an overflowing node breaks apart and pushes a key upward.

Cascading split

A split that causes the parent to overflow, forcing another split upward.

Black depth

The number of black nodes on a path from the root to a null child.

In Phase 4, black depth matches the idea that all 2-3-4 leaves are at the same level.

23. Phase 4 Final Summary

Phase 4 connects 2-3-4 Trees and Red-Black Trees.

The main idea is:

A Red-Black Tree is a 2-3-4 Tree drawn as a binary tree.

A 2-3-4 Tree stores multiple keys in one node.

A Red-Black Tree stores one key per node, but red nodes show grouped keys.

The core mapping is:

Black node = separate 2-3-4 node
Red node = extra key inside black parent's 2-3-4 node

A 2-node maps to:

[20] -> 20B

A 3-node maps to:

[20, 40] -> black node with one red child

A 4-node maps to:

[10, 20, 30] -> black 20 with red 10 and red 30

Recoloring means:

2-3-4 splitting

Rotation means:

binary shape rearrangement

New Red-Black nodes start red because red means:

try to add this key into an existing 2-3-4 node

The most important final memory sentence is:

Red-Black Trees and 2-3-4 Trees are the same balancing idea shown in two different forms.

24. Mini Practice Questions

Question 1

What 2-3-4 node does this Red-Black shape represent?

20B

Answer

[20]

Reason

A single black node represents one 2-node.

Question 2

What 2-3-4 node does this represent?

20B

40R

Answer

[20, 40]

Reason

The red child is grouped with the black parent.

Question 3

What 2-3-4 node does this represent?

20B
/
10R 30R

Answer

[10, 20, 30]

Reason

The black middle key and two red children form one 4-node.

Question 4

In Red-Black insertion, what does recoloring usually mean in 2-3-4 language?

Answer

Splitting an overflowing 2-3-4 node.

Question 5

In Red-Black insertion, what does rotation usually mean?

Answer

Rearranging the binary drawing while keeping sorted order.

Question 6

Why do new Red-Black nodes start red?

Answer

Because red means the new key is first treated as an extra key inside an existing 2-3-4 node.

Question 7

What does a black child usually mean in the 2-3-4 mapping?

Answer

It usually starts a separate 2-3-4 node.

Question 8

What is the Red-Black form of this 2-3-4 node?

[10, 20, 30]

Answer

20B
/
10R 30R

Question 9

Does rotation change the sorted order of keys?

Answer

No.

Rotation changes the shape, but the BST sorted order stays the same.

Question 10

What is the one-sentence summary of Phase 4?

Answer

A Red-Black Tree is a 2-3-4 Tree drawn as a binary tree, where black nodes start real 2-3-4 nodes and red nodes are extra keys inside those nodes.

25. One-Page Memory Sheet

Phase 4: Red-Black Trees and 2-3-4 Trees

Main idea:

Red-Black Tree = 2-3-4 Tree drawn as a binary tree

Core mapping:

Black node = separate 2-3-4 node
Red node = extra key grouped with black parent

2-node:

[20]
=
20B

3-node:

[20, 40]
=
20B

40R

4-node:

[10, 20, 30]
=
20B
/
10R 30R

Insertion meaning:

New red node = try to add key into existing 2-3-4 node

Repair meaning:

Red uncle -> recolor -> 2-3-4 split
Black/null uncle -> rotate -> rearrange binary shape

Important rules:

Red is allowed.
Red-red is not allowed.
Black nodes count as real 2-3-4 levels.
Same black depth matches all 2-3-4 leaves staying level.
Rotation keeps sorted order.
Recoloring pushes balance upward.

Common mistakes:

Do not think red is always bad.
Do not treat a red child as a separate 2-3-4 node.
Do not forget that a 4-node has 3 keys, not 4 keys.
Do not think recoloring and rotation are random.
Do not forget that Red-Black Trees are still BSTs.

Final memory sentence:

A Red-Black Tree is a 2-3-4 Tree in binary form: black nodes are real nodes, and red nodes are extra keys grouped inside them.

26. Final One-Sentence Summary

Phase 4 shows that Red-Black Trees and 2-3-4 Trees are the same balanced search-tree idea, where recoloring acts like splitting and rotation acts like rearranging the binary drawing of grouped keys.